



Cours BTS Électronique

Raspberry Pi · Intelligence Artificielle · Internet des Objets

Chapitres 4, 5 & 6 – Programme Complet

📖 Table des Matières

Chap. 4 – Raspberry Pi : Architecture, Linux, GPIO, Serveur LAMP

4.1 Intro · 4.2 Architecture · 4.3 Linux · 4.4 Installation · 4.5 GPIO · 4.6 Serveur · 4.7 Projets

Chap. 5 – Intelligence Artificielle : ML, Deep Learning, TinyML

5.1–5.8 Fondamentaux · 5.9 Python · 5.10 TensorFlow · 5.11 TinyML · 5.12 Projets

Chap. 6 – Internet des Objets : ESP32, MQTT, Node-RED, Android

6.1–6.7 Fondamentaux · 6.8 Node-RED · 6.9 Dashboards · 6.10 Android · 6.11 App Inventor · 6.12 Projet

Lycée Moulay Youssef – Tanger · BTS Électronique

🍷 Chapitre 4 - Raspberry Pi

Architecture · Linux · GPIO · Python · Serveur Web

🔧 4.1 - Qu'est-ce que le Raspberry Pi ?

Définition, historique et applications

🎯 Objectifs

- ✓ Comprendre ce qu'est un Raspberry Pi et son rôle en électronique
- ✓ Connaître l'historique et les différents modèles
- ✓ Identifier les domaines d'application

► Définition

Le **Raspberry Pi** est un **ordinateur monocarte** (SBC - Single Board Computer) de la taille d'une carte de crédit, créé par la **Raspberry Pi Foundation** (Royaume-Uni). Conçu initialement pour l'enseignement de l'informatique, il est aujourd'hui utilisé par des millions de personnes pour des projets DIY, serveurs, domotique et IoT.

📏 Dimensions et Prix

Taille : 85 × 56 mm (carte de crédit) · **Poids** : 45 grammes · **Prix** : 35€ - 95€ selon modèle

💡 Le saviez-vous ?

Plus de 50 millions d'unités vendues depuis 2012, faisant du Raspberry Pi l'ordinateur britannique le plus vendu de l'histoire !

► Historique

Année	Modèle	Caractéristiques
2012	Pi 1 Model B	256 MB RAM, ARM11, première version
2015	Pi 2	4× plus rapide, quad-core ARM Cortex-A7
2016	Pi 3	WiFi et Bluetooth intégrés
2019	Pi 4	Jusqu'à 8 GB RAM, USB 3.0, 4K
2023	Pi 5	3× plus rapide, PCIe, nouveau SoC

► Applications

Le Raspberry Pi est utilisé dans de nombreux domaines :

- **Éducation et formation** - Enseignement de la programmation et de l'électronique
- **Serveurs web/médias** - Apache, Nginx, Plex, Kodi
- **Domotique intelligente** - Home Assistant, contrôle de la maison
- **Stations météo IoT** - Capteurs connectés avec dashboards
- **Retrogaming** - RetroPie pour émuler les consoles rétro
- **Robotique** - Contrôle de robots, drones, véhicules autonomes

🔧 4.2 - Architecture Matérielle du Raspberry Pi 4

Processeur, mémoire, connectivité et GPIO

► Processeur (SoC)

Broadcom BCM2711 - Quad-core ARM Cortex-A72 à 1.5 GHz (1.8 GHz sur Pi 5). Architecture 64-bit ARMv8 avec GPU VideoCore VI intégré. Ce SoC offre des performances suffisantes pour un usage bureautique, serveur et développement embarqué.

► Mémoire et Stockage

RAM : LPDDR4-3200 disponible en 2, 4 ou 8 GB · **Stockage** : carte microSD (8 GB minimum, classe 10 A1/A2 recommandée). Pour des performances optimales, un SSD via USB 3.0 est conseillé.

► Connectivité

Interface	Spécification
USB	2× USB 3.0 + 2× USB 2.0
Vidéo	2× micro-HDMI 4K@60Hz
Réseau	Gigabit Ethernet (vrai gigabit)
WiFi	WiFi 5 (802.11ac) dual-band 2.4/5 GHz
Bluetooth	5.0 BLE (Bluetooth Low Energy)
Audio	Jack 3.5 mm + sortie audio HDMI
Alimentation	USB-C 5V/3A (15W) - Conso : 2.7W idle, 6.4W charge

► Comparaison des Modèles

Modèle	CPU	RAM	WiFi	Prix	Usage
Pi Zero 2 W	Quad-core 1GHz	512 MB	Oui	~15€	Projets compacts
Pi 4 Model B	Cortex-A72 1.5GHz	2/4/8 GB	Oui	35-75€	Meilleur rapport Q/P
Pi 5	Cortex-A76 2.4GHz	4/8 GB	Oui	60-95€	Performances max

💡 Conseil :

Pi 4 4GB = meilleur rapport qualité/prix. Pi 5 = performances maximales. Pi Zero 2 = projets compacts et embarqués.

👤 4.3 - Linux : Introduction et Commandes Essentielles

Système d'exploitation, navigation et gestion de fichiers

► Qu'est-ce que Linux ?

Linux est un système d'exploitation libre et open-source basé sur Unix. **Raspberry Pi OS** (anciennement Raspbian) est une distribution Debian optimisée pour le Pi. Linux est gratuit, léger (fonctionne avec 512 MB RAM), très stable, sécurisé, et dispose d'une immense communauté avec des milliers de logiciels gratuits.

📁 Structure des Répertoires

Répertoire	Description
/	Racine du système
/home	Dossiers utilisateurs
/bin	Commandes binaires essentielles
/etc	Fichiers de configuration système
/var	Données variables (logs, cache)
/tmp	Fichiers temporaires
/usr	Applications et utilitaires

⚠️**Important :**
Linux est sensible à la casse ! "Documents" ≠ "documents"

►Commandes de Navigation

```
# Afficher le répertoire actuel
$ pwd
/home/pi

# Changer de répertoire
$ cd Documents      # Aller dans Documents
$ cd ..             # Remonter d'un niveau
$ cd ~              # Aller au répertoire home

# Lister les fichiers
$ ls -la            # Liste détaillée + fichiers cachés
```

🔍 Astuces Navigation

Tab : complétion automatique · **↑ ↓** : historique commandes · **Ctrl+C** : annuler commande · **clear** : nettoyer écran · **~** : répertoire home (/home/pi)

►Gestion de Fichiers

```
# Créer fichiers et dossiers
$ mkdir projet      # Créer un dossier
$ touch fichier.txt # Créer un fichier vide
$ nano fichier.txt   # Éditer avec nano

# Copier et déplacer
$ cp src.txt dest.txt # Copier
$ cp -r dossier/ backup/# Copier récursivement
$ mv old.txt new.txt  # Renommer/déplacer

# Supprimer
$ rm fichier.txt      # Supprimer fichier

# Voir le contenu
$ cat fichier.txt     # Afficher tout
$ head -n 10 fichier.txt# 10 premières lignes
$ tail -f log.txt     # Suivre en temps réel
```

⚠️**DANGER :**
`rm -rf /` supprime tout le système ! Toujours vérifier avant d'utiliser `rm -rf .`

►Commandes Système

```
# Mise à jour système
$ sudo apt update      # Rafraîchir les paquets
$ sudo apt upgrade -y  # Mettre à jour
$ sudo apt dist-upgrade # Mise à jour complète

# Redémarrage et arrêt
$ sudo reboot          # Redémarrer
$ sudo shutdown -h now # Éteindre

# Réseau
$ ifconfig             # Infos réseau
$ hostname -I          # Adresse IP
$ ping google.com      # Tester connectivité

# Informations système
$ df -h               # Espace disque
$ free -h             # Mémoire RAM
$ top                 # Processus en cours
```



4.4 - Installation et Configuration de Raspberry Pi OS

Installation, premier boot, SSH et VNC

►Installation de Raspberry Pi OS

L'outil officiel **Raspberry Pi Imager** permet d'installer facilement le système sur carte microSD.

📦 **Prérequis**

Carte microSD 8 GB minimum · Lecteur de carte · Connexion Internet · PC Windows/Mac/Linux

🔧 **Étapes d'Installation**

1. **Télécharger** Raspberry Pi Imager depuis raspberrypi.com
2. **Choisir l'OS** : Raspberry Pi OS 64-bit (recommandé)
3. **Sélectionner** la carte SD comme destination
4. **Configurer** (icône engrenage) : WiFi, hostname, SSH, mot de passe
5. **Écrire** et attendre la fin du processus

►Configuration Initiale - raspi-config

```
$ sudo raspi-config
```

Cet outil en mode texte permet de configurer : langue et clavier, WiFi et réseau, SSH / VNC, overclocking, interfaces (I2C, SPI, UART).

📋 **Checklist Premier Boot**

- ✓ Changer le mot de passe par défaut (`passwd`)
- ✓ Configurer le WiFi
- ✓ Effectuer la mise à jour système (`sudo apt update && sudo apt upgrade -y`)
- ✓ Activer SSH pour l'accès distant

- ✓ Configurer le fuseau horaire (timezone)

►SSH - Accès Terminal à Distance

SSH (Secure Shell) est un protocole crypté pour accéder au terminal du Pi depuis un autre ordinateur du réseau.

```
# Activer SSH
$ sudo raspi-config          # Interface Options → SSH → Enable

# Se connecter depuis un autre PC
$ ssh pi@192.168.1.100      # Port 22 par défaut
```

Windows : PuTTY ou PowerShell · **Mac/Linux** : Terminal natif

►VNC - Bureau à Distance

VNC (Virtual Network Computing) permet d'accéder à l'interface graphique complète du Pi depuis n'importe quel ordinateur.

```
# Activer VNC
$ sudo raspi-config          # Interface → VNC → Enable
```

Client : Télécharger RealVNC Viewer · **Connexion** : 192.168.1.100:5900 · **Login** : pi / votre_mot_de_passe

🔌 **VNC**
= interface graphique complète (souris + clavier).
SSH
= ligne de commande uniquement. Utiliser SSH pour les tâches serveur, VNC pour le développement graphique.

🔌 4.5 - GPIO : Entrées/Sorties Programmables

Connecter et programmer des composants électroniques

►Introduction au GPIO

Le **GPIO** (General Purpose Input/Output) est un connecteur de **40 broches** permettant de connecter des composants électroniques : LED, capteurs, moteurs, écrans, relais, etc. C'est l'interface entre le monde numérique (programmation) et le monde physique (électronique).

📊 Répartition des 40 Broches

- **26 GPIO** programmables (entrée/sortie numérique)
- **2 broches 5V** (pins 2, 4) et **2 broches 3.3V** (pins 1, 17)
- **8 broches GND** (masse) : pins 6, 9, 14, 20, 25, 30, 34, 39
- **Protocoles** : I2C (GPIO 2, 3), SPI (GPIO 7-11), UART (GPIO 14, 15), PWM (GPIO 12, 13)

⚠ **SÉCURITÉ CRITIQUE :**
Tension max =
3.3V
. Courant max =
16 mA/pin
. Toujours débrancher avant câblage. Utiliser des résistances.
Ne JAMAIS connecter 5V aux GPIO !

```
# Afficher le schéma des broches
$ pinout
```

Référence complète : **pinout.xyz**

►Programmation GPIO avec Python (GPIOZero)

GPIOZero est une bibliothèque Python simple et puissante pour contrôler les GPIO du Raspberry Pi.

```
# Installation
$ sudo apt install python3-gpiozero
```

🔌 Exemple 1 : LED Clignotante

```
from gpiozero import LED
from time import sleep

led = LED(17)    # LED connectée au GPIO 17

while True:
    led.on()      # Allumer
    sleep(1)
    led.off()     # Éteindre
    sleep(1)
```

🔌 Exemple 2 : Bouton + LED

```
from gpiozero import LED, Button

led = LED(17)
btn = Button(2)    # Bouton sur GPIO 2

btn.when_pressed = led.on
btn.when_released = led.off
```

🔌 Exemple 3 : Capteur Température DHT22

```
import Adafruit_DHT

sensor = Adafruit_DHT.DHT22
pin = 4    # GPIO 4

humidity, temp = Adafruit_DHT.read_retry(sensor, pin)
print(f"Temp: {temp:.1f}°C Humidité: {humidity:.1f}%")
```

🌐 4.6 - Serveur Web et Stack LAMP

Apache, PHP, MySQL sur Raspberry Pi

►Serveur Web Apache

Apache est le serveur web le plus populaire au monde. Il permet de transformer votre Pi en serveur web accessible depuis le réseau local ou Internet.

```
# Installation
$ sudo apt install apache2 -y
$ sudo systemctl enable apache2

# Gestion du service
$ sudo systemctl start apache2
$ sudo systemctl restart apache2

# Tester : ouvrir http://localhost ou http://IP-du-Pi
# Répertoire des fichiers web : /var/www/html/
```

► PHP et MySQL (Stack LAMP)

```
# Installer PHP
$ sudo apt install php libapache2-mod-php -y

# Tester PHP
$ echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
# Ouvrir : http://IP/info.php

# Installer MySQL (MariaDB)
$ sudo apt install mariadb-server -y
$ sudo mysql_secure_installation
```

🔧 Stack LAMP complet !

Vous avez maintenant Linux + Apache + MySQL + PHP sur votre Pi. Vous pouvez héberger des sites web, des APIs REST, et des applications IoT.



4.7 - Projets et Ressources

Idées de projets et documentation

► Projets Pratiques GPIO

Projet	Matériel	GPIO	Difficulté
🔦 LED Clignotante	1 LED + résistance 330Ω	Pin 17	★
🔘 Bouton Interrupteur	1 bouton poussoir	Pin 2	★
🌡️ Capteur Température	DHT22	Pin 4	★★
✂️ Capteur Distance	HC-SR04 ultrasonique	Pin 23, 24	★★

► Idées de Projets Avancés

- 🎮 **RetroPie** - Console retrogaming avec émulateurs (NES, SNES, N64, PSX)
- 📹 **Caméra Surveillance** - Motion + Pi Camera Module pour vidéosurveillance
- 🏠 **Domotique** - Home Assistant pour contrôler toute la maison
- 💻 **NAS Personnel** - Stockage réseau avec Samba et disques externes
- 🌐 **Pi-hole** - Bloqueur de publicités DNS pour tout le réseau
- 🌤️ **Station Météo** - Capteurs + Dashboard web en temps réel

► Ressources Officielles

- 📄 **Documentation** : raspberrypi.com/documentation
- 💬 **Forums** : forums.raspberrypi.com
- 📖 **GPIOZero** : gpiozero.readthedocs.io
- 🔌 **Pinout** : pinout.xyz

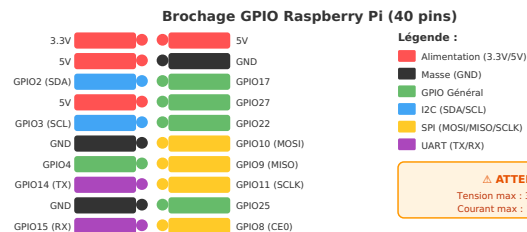


Figure - Brochage GPIO du Raspberry Pi (extrait 20 pins / 40)

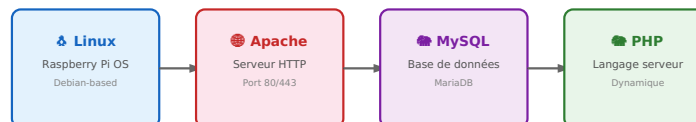


Figure - Architecture LAMP (Linux, Apache, MySQL, PHP)

Chapitre 5 - Intelligence Artificielle

Machine Learning · Deep Learning · TinyML

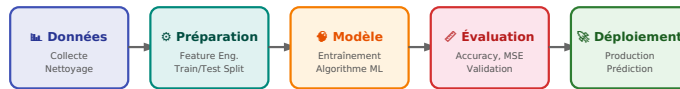


Figure – Pipeline complet du Machine Learning

5.5.1 – Introduction a l'Intelligence Artificielle

Concepts fondamentaux et historique de l'IA

Objectifs du chapitre

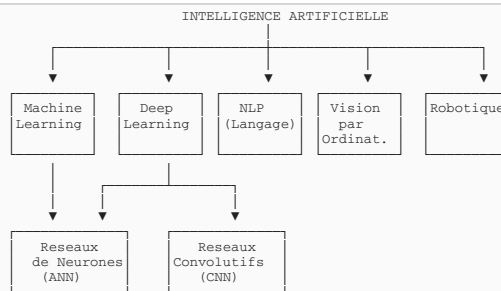
- ✓ Définir l'Intelligence Artificielle et ses branches
- ✓ Connaître l'historique et les évolutions de l'IA
- ✓ Distinguer IA faible et IA forte
- ✓ Identifier les applications de l'IA en électronique

►5.1.1 Définition de l'Intelligence Artificielle

Intelligence Artificielle (IA)

L'Intelligence Artificielle est un domaine de l'informatique qui vise à créer des systèmes capables d'effectuer des tâches nécessitant normalement l'intelligence humaine : apprentissage, raisonnement, perception, compréhension du langage, prise de décision.

Branches de l'Intelligence Artificielle



►5.1.2 Historique de l'IA

Annee	Evenement	Importance
1943	McCulloch & Pitts : Neurone artificiel	Premier modele mathematique de neurone
1950	Test de Turing	Critere d'intelligence machine
1956	Conference de Dartmouth	Naissance officielle du terme "IA"
1957	Perceptron (Rosenblatt)	Premier reseau de neurones
1986	Retropropagation	Entraînement efficace des reseaux
1997	Deep Blue bat Kasparov	IA bat champion d'echecs
2012	AlexNet (ImageNet)	Revolution du Deep Learning
2016	AlphaGo bat Lee Sedol	IA maitrise le jeu de Go
2022	ChatGPT (OpenAI)	IA conversationnelle grand public

►5.1.3 IA Faible vs IA Forte



IA Faible (Narrow AI)

Conçue pour une tâche spécifique. Ex: reconnaissance faciale, traduction, recommandation. C'est l'IA actuelle.



IA Forte (General AI)

Intelligence comparable à l'humain, capable de raisonner, apprendre et s'adapter à toute tâche. N'existe pas encore.

►5.1.4 Applications de l'IA en Electronique



Maintenance Predictive

Détection de pannes avant qu'elles surviennent



Contrôle Qualité

Inspection automatique des circuits



Reconnaissance Vocale

Commandes vocales pour IoT



Vision par Ordinateur

Détection d'objets, de gestes



Optimisation Énergie

Gestion intelligente de la consommation



Robotique

Navigaton autonome, manipulation



Figure - Les 3 types d'apprentissage automatique

5.2 - Types d'Apprentissage Automatique

Supervise, non supervise et par renforcement

Objectifs du chapitre

- ✓ Distinguer les trois types d'apprentissage machine
- ✓ Identifier les cas d'usage de chaque type
- ✓ Comprendre les notions de features et labels
- ✓ Connaître les algorithmes principaux de chaque categorie

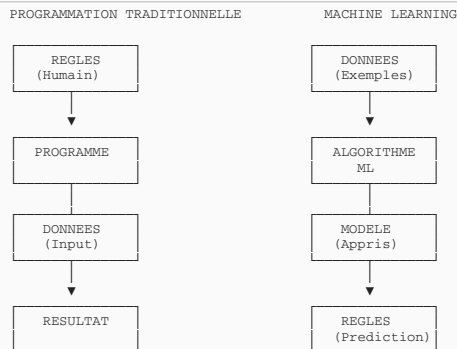
5.2.1 Machine Learning : Vue d'ensemble

Machine Learning (Apprentissage Automatique)

Sous-domaine de l'IA ou les algorithmes

apprennent à partir des données

sans être explicitement programmés pour chaque cas. Le système s'améliore avec l'expérience.



5.2.2 Apprentissage Supervisé

L'algorithme apprend à partir de données **étiquetées** (avec les réponses connues).

APPRENTISSAGE SUPERVISE

Données d'entraînement:

Features (X)	Label (Y)
[taille, poids, age]	"Malade"
[taille, poids, age]	"Sain"
[taille, poids, age]	"Malade"
...	...



MODELE ML (Training)

← Apprentissage

Nouvelles données:

[taille, poids, age] → Modele → Prediction: "Sain"

Types de problèmes supervisés

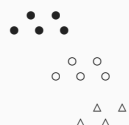
Type	Sortie	Exemples	Algorithmes
Classification	Catégorie discrete	Spam/Non-spam, Malade/Sain	KNN, SVM, Arbres de decision
Regression	Valeur continue	Prix maison, temperature	Regression lineaire, polynomiale

5.2.3 Apprentissage Non Supervisé

L'algorithme découvre des **structures cachées** dans les données sans étiquettes.

APPRENTISSAGE NON SUPERVISE (Clustering)

Données sans étiquettes:



Résultat (groupes découverts):



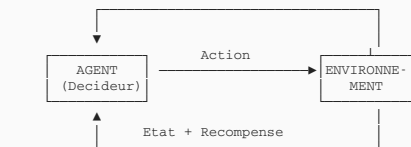
Applications

- **Clustering** : Segmentation clients, regroupement documents
- **Reduction de dimensions** : PCA, compression de données
- **Detection d'anomalies** : Fraudes, intrusions réseau

5.2.4 Apprentissage par Renforcement

Un **agent** apprend en interagissant avec un environnement, recevant des **recompenses** ou **penalités**.

APPRENTISSAGE PAR RENFORCEMENT



Exemples:

- Jeux (AlphaGo, echecs)
- Robotique (navigation)
- Vehicules autonomes

►5.2.5 Resume des Types d'Apprentissage

Type	Donnees	Objectif	Exemple IoT
Supervise	Etiquetees	Predire une sortie	Detection de chute (accelerometre)
Non supervise	Non etiquetees	Decouvrir des patterns	Detection d'anomalies capteurs
Renforcement	Recompenses	Optimiser une strategie	Regulation thermostat intelligent



5.5.3 - Preparation des Donnees

Collecte, nettoyage et transformation des donnees

🎯 Objectifs du chapitre

- ✓ Comprendre l'importance de la qualite des donnees
- ✓ Maitriser les techniques de pretraitement
- ✓ Normaliser et standardiser les donnees
- ✓ Diviser les donnees en ensembles d'entrainement et de test

►5.3.1 Importance des Donnees

⚠ Garbage In, Garbage Out

La qualite du modele depend directement de la qualite des donnees. Un mauvais jeu de donnees produira un mauvais modele, peu importe l'algorithme utilise.

Pipeline de donnees typique



►5.3.2 Nettoyage des Donnees

Problemes courants et solutions

- **Valeurs manquantes** : Supprimer, imputer (moyenne, mediane, mode)
- **Doublons** : Identifier et supprimer les lignes identiques
- **Outliers (valeurs aberrantes)** : Detecter (IQR, Z-score), supprimer ou corriger
- **Incoherences** : Standardiser les formats (dates, unites)

```

# Nettoyage avec Pandas en Python
import pandas as pd
import numpy as np

# Charger les donnees
df = pd.read_csv('capteurs.csv')

# Verifier les valeurs manquantes
print(df.isnull().sum())

# Remplir les valeurs manquantes par la moyenne
df['temperature'].fillna(df['temperature'].mean(), inplace=True)

# Supprimer les doublons
df.drop_duplicates(inplace=True)

# Detecter les outliers avec IQR
Q1 = df['temperature'].quantile(0.25)
Q3 = df['temperature'].quantile(0.75)
IQR = Q3 - Q1
df = df[(df['temperature'] >= Q1 - 1.5*IQR) &
        (df['temperature'] <= Q3 + 1.5*IQR)]
  
```

►5.3.3 Normalisation et Standardisation

Normalisation Min-Max

$$X' = (X - X_{min}) / (X_{max} - X_{min})$$

Ramene les valeurs entre 0 et 1

Standardisation (Z-Score)

$$X' = (X - \mu) / \sigma$$

Moyenne = 0, Ecart-type = 1

```

# Normalisation et Standardisation avec Scikit-Learn
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Normalisation Min-Max (0-1)
scaler_minmax = MinMaxScaler()
X_normalized = scaler_minmax.fit_transform(X)

# Standardisation (Z-score)
scaler_std = StandardScaler()
X_standardized = scaler_std.fit_transform(X)
  
```

►5.3.4 Division Train/Test

Les donnees sont divisees pour **entrainer** le modele et **evaluer** sa performance sur des donnees inconnues.

DATASET COMPLET (100%)

TRAIN SET (70-80%) Apprentissage du modele	TEST SET (20-30%) Evaluation finale
---	--

Optionnel: Validation Set (pour tuning hyperparametres)
Train: 60% | Validation: 20% | Test: 20%

```
# Division des donnees avec Scikit-Learn
from sklearn.model_selection import train_test_split

# X = features, y = labels
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% pour le test
    random_state=42,    # Pour reproductibilite
    stratify=y          # Maintenir la proportion des classes
)

print(f"Train: {len(X_train)}, Test: {len(X_test)}")
```



5.5.4 - Algorithmes de Regression

Prediction de valeurs continues

🎯 Objectifs du chapitre

- ✓ Comprendre le principe de la regression
- ✓ Implementer une regression lineaire simple et multiple
- ✓ Evaluer les performances avec MSE, RMSE, R²
- ✓ Appliquer la regression a des donnees de capteurs

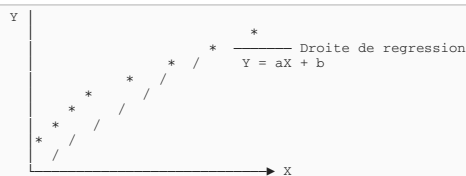
►5.4.1 Regression Lineaire Simple

Modelise la relation lineaire entre une variable d'entree X et une sortie Y.

Equation de la droite de regression

$$Y = aX + b$$

a = pente (coefficient), b = ordonnee a l'origine (intercept)



```
# Regression lineaire simple avec Scikit-Learn
from sklearn.linear_model import LinearRegression
import numpy as np

# Donnees: temperature vs consommation energetique
X = np.array([[15], [18], [22], [25], [30], [35]]) # Temperature °C
y = np.array([120, 135, 150, 165, 190, 220])      # Consommation W

# Creer et entrainer le modele
model = LinearRegression()
model.fit(X, y)

# Coefficients
print(f"Pente (a): {model.coef_[0]:.2f}")
print(f"Intercept (b): {model.intercept_: .2f}")

# Prediction pour 28°C
prediction = model.predict([[28]])
print(f"Consommation a 28°C: {prediction[0]:.1f} W")
```

►5.4.2 Regression Lineaire Multiple

Plusieurs variables d'entree pour predire une sortie :

Regression multiple

$$Y = a_1X_1 + a_2X_2 + \dots + a_nX_n + b$$

Plusieurs features contribuent a la prediction

```
# Regression multiple: predire consommation selon temp, humidite, heure
X = np.array([
    [20, 60, 8],   # [temperature, humidite%, heure]
    [25, 55, 12],
    [30, 70, 14],
    [22, 65, 18],
    [28, 50, 20]
])
y = np.array([150, 180, 220, 160, 200]) # Consommation W

model = LinearRegression()
model.fit(X, y)

# Coefficients pour chaque feature
print("Coefficients:", model.coef_)
# Ex: [5.2, 0.8, 2.1] → temp a plus d'impact que humidite
```

►5.4.3 Metriques d'Evaluation

Metrique	Formule	Interpretation
MSE (Mean Squared Error)	$\sum (y - \hat{y})^2 / n$	Erreur quadratique moyenne. Plus petit = meilleur
RMSE (Root MSE)	$\sqrt{\text{MSE}}$	Meme unite que Y. Interpretable
MAE (Mean Absolute Error)	$\sum y - \hat{y} / n$	Erreur absolue moyenne

R² (Coefficient determination)	1 - SS_res/SS_tot	0-1. Proche de 1 = bon modele
<pre># Evaluation du modele from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score import numpy as np y_pred = model.predict(X_test) mse = mean_squared_error(y_test, y_pred) rmse = np.sqrt(mse) mae = mean_absolute_error(y_test, y_pred) r2 = r2_score(y_test, y_pred) print(f"MSE: {mse:.2f}") print(f"RMSE: {rmse:.2f}") print(f"MAE: {mae:.2f}") print(f"R²: {r2:.3f}")</pre>		



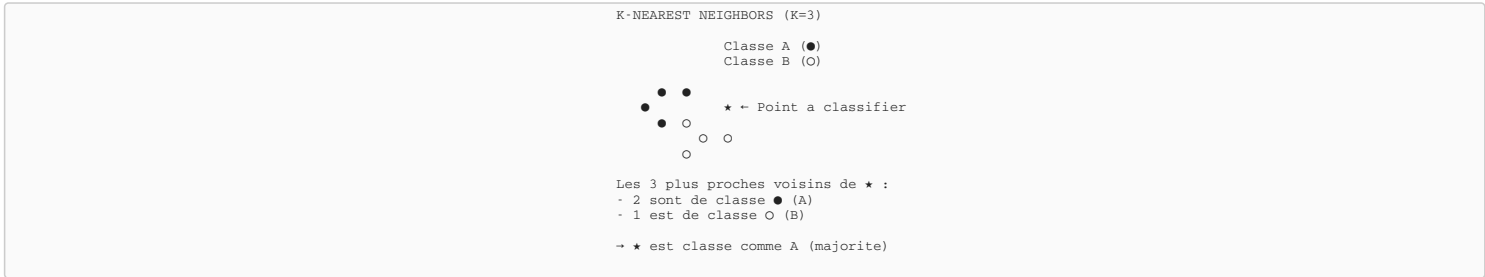
5.5.5 - Algorithmes de Classification

Prediction de categories

- 🎯 Objectifs du chapitre
- ✓ Comprendre la difference classification vs regression
 - ✓ Implementer K-Nearest Neighbors (KNN)
 - ✓ Implementer la regression logistique
 - ✓ Evaluer avec accuracy, precision, recall, F1-score

►5.5.1 K-Nearest Neighbors (KNN)

Classe un point selon la **majorite de ses K plus proches voisins**.



```
# Classification KNN
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Donnees: [temperature, humidite] → Etat (0=Normal, 1=Alerte)
X = np.array([[22, 45], [25, 50], [28, 55], [35, 80],
              [38, 85], [40, 90], [20, 40], [42, 95]])
y = np.array([0, 0, 0, 1, 1, 1, 0, 1])

# Division train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)

# Creer et entrainer KNN (K=3)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Prediction
y_pred = knn.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")

# Classifier un nouveau point
new_point = [[36, 75]]
prediction = knn.predict(new_point)
print(f"Prediction: {'Alerte' if prediction[0] else 'Normal'}")
```

►5.5.2 Regression Logistique

Malgre son nom, c'est un algorithme de **classification binaire**. Utilise la fonction sigmoide.

Fonction Sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Transforme une valeur en probabilite entre 0 et 1

```
# Regression logistique
from sklearn.linear_model import LogisticRegression

# Creer et entrainer
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

# Prediction
y_pred = log_reg.predict(X_test)

# Probabilites
proba = log_reg.predict_proba([[36, 75]])
print(f"Probabilite Normal: {proba[0][0]:.2%}")
print(f"Probabilite Alerte: {proba[0][1]:.2%}")
```

►5.5.3 Metriques de Classification

Matrice de Confusion

		PREDICTION		
		Positif	Negatif	
REEL	Positif	TP	FN	TP = True Positive (Vrais Positifs) FP = False Positive (Faux Positifs) TN = True Negative (Vrais Negatifs) FN = False Negative (Faux Negatifs)
	Negatif	FP	TN	

Metrique	Formule	Signification
Accuracy	$(TP+TN) / \text{Total}$	% de predictions correctes
Precision	$TP / (TP+FP)$	Parmi les positifs predits, % de vrais
Recall	$TP / (TP+FN)$	Parmi les vrais positifs, % detectes
F1-Score	$2 \times (P \times R) / (P + R)$	Moyenne harmonique Precision/Recall

```

# Metriques de classification
from sklearn.metrics import confusion_matrix, classification_report

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
print("Matrice de confusion:")
print(cm)

# Rapport complet
print(classification_report(y_test, y_pred))

```



5.5.6 - Arbres de Decision

Algorithmes interpretables pour classification et regression

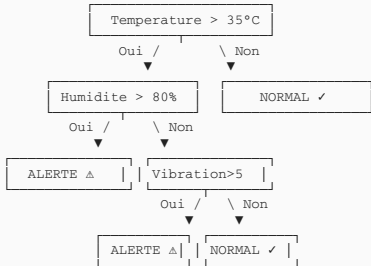
🎯 Objectifs du chapitre

- ✓ Comprendre le fonctionnement des arbres de decision
- ✓ Implementer un arbre de decision
- ✓ Visualiser et interpreter l'arbre
- ✓ Decouvrir Random Forest (foret aleatoire)

►5.6.1 Principe de l'Arbre de Decision

L'arbre de decision pose une serie de **questions binaires** pour arriver a une decision.

ARBRE DE DECISION - Detection d'anomalie capteur



►5.6.2 Implementation avec Scikit-Learn

```

# Arbre de decision pour classification
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Donnees: [temperature, humidite, vibration] -> Etat
X = np.array([
    [25, 50, 2], [30, 60, 3], [38, 85, 4],
    [40, 90, 6], [22, 45, 1], [42, 95, 8]
])
y = np.array([0, 0, 1, 1, 0, 1]) # 0=Normal, 1=Alerte

# Creer et entrainer l'arbre
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X, y)

# Visualiser l'arbre
plt.figure(figsize=(12, 8))
plot_tree(tree, feature_names=['Temp', 'Hum', 'Vib'],
          class_names=['Normal', 'Alerte'], filled=True)
plt.show()

# Prediction
new_data = [[36, 82, 5]]
print(f"Prediction: {tree.predict(new_data)}")

```

►5.6.3 Random Forest (Foret Aleatoire)

Ensemble de plusieurs arbres de decision qui **votent** pour la decision finale. Plus robuste qu'un seul arbre.

```

# Random Forest
from sklearn.ensemble import RandomForestClassifier

# Creer le modele avec 100 arbres
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Evaluation
accuracy = rf.score(X_test, y_test)
print(f"Accuracy: {accuracy:.2%}")

# Importance des features
print("Importance des features:")
for name, importance in zip(['Temp', 'Hum', 'Vib'], rf.feature_importances_):
    print(f" {name}: {importance:.2%}")

```



5.5.7 - Reseaux de Neurones Artificiels

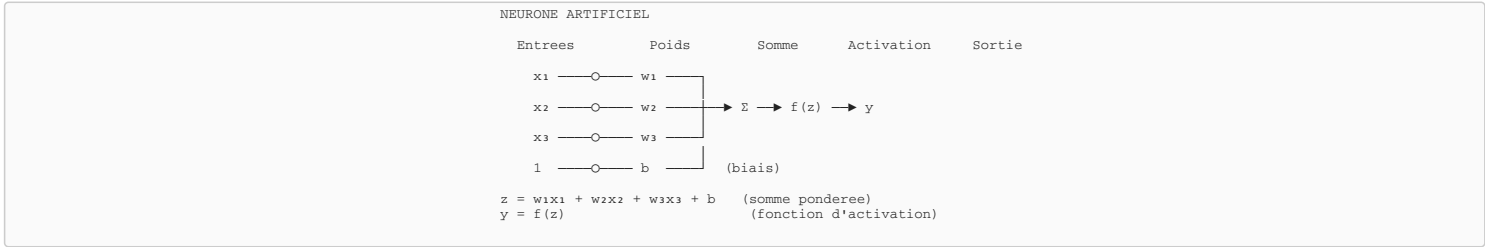
Fondements du Deep Learning

🎯 Objectifs du chapitre

- ✓ Comprendre le neurone artificiel (perceptron)
- ✓ Connaitre les fonctions d'activation
- ✓ Comprendre l'architecture multicouche
- ✓ Expliquer la retropropagation

►5.7.1 Le Neurone Artificiel (Perceptron)

5.7.2 Le Neuron Artificiel (Perceptron)



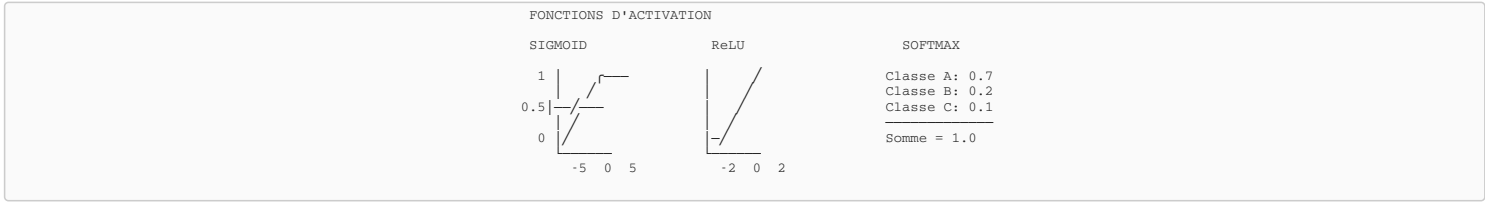
Calcul du neurone

$y = f(\Sigma w_i x_i + b)$

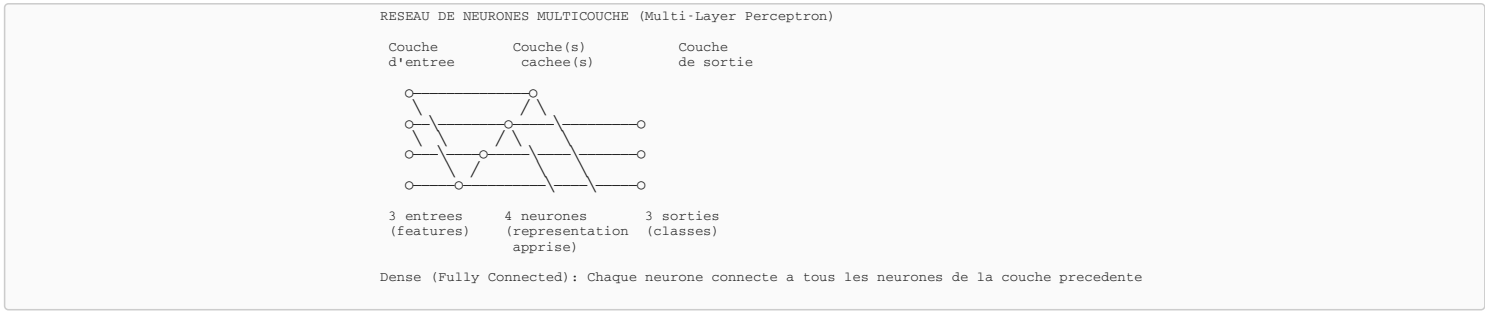
Somme ponderee des entrees + biais, passee par une activation

5.7.2 Fonctions d'Activation

Fonction	Formule	Sortie	Usage
Sigmoid	$1 / (1 + e^{-x})$	[0, 1]	Classification binaire (sortie)
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	[-1, 1]	Couches cachees
ReLU	$\max(0, x)$	$[0, +\infty)$	Couches cachees (le plus utilise)
Softmax	$e^{x_i} / \Sigma e^{x_j}$	$[0, 1]$, somme=1	Classification multi-classes



5.7.3 Architecture Multicouche (MLP)



5.7.4 Entrainement et Retropropagation

CYCLE D'ENTRAÎNEMENT

1. FORWARD PROPAGATION (Propagation avant)

Input → Hidden → Output → Prediction ($\hat{y}$)

2. CALCUL DE L'ERREUR (Loss)

$Loss = f(y_{vrai}, \hat{y})$ ex: MSE, Cross-Entropy

3. BACKPROPAGATION (Retropropagation)

← Calcul des gradients: $\partial Loss / \partial w$ pour chaque poids

4. MISE A JOUR DES POIDS (Gradient Descent)

$w_{nouveau} = w_{ancien} - \eta \times \partial Loss / \partial w$
(η = learning rate, taux d'apprentissage)

5. REPETER pour chaque batch/epoch

🔧 Epoch et Batch

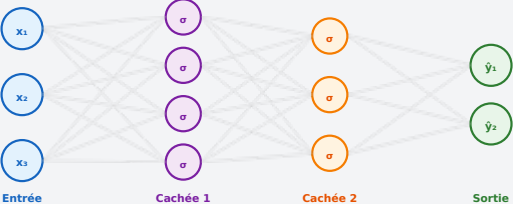
-

Epoch:
Un passage complet sur toutes les donnees d'entrainement

-

Batch:
Sous-ensemble de donnees traitees ensemble avant mise a jour des poids





Entrée

Cachée 1

Cachée 2

Sortie

Figure - Réseau de neurones multicouche (MLP) avec fonctions d'activation σ

5.8 - Deep Learning

Reseaux de neurones profonds et architectures avancees

- 🎯 **Objectifs du chapitre**
- ✓ Comprendre ce qu'est le Deep Learning
 - ✓ Découvrir les réseaux convolutifs (CNN)
 - ✓ Connaître les réseaux récurrents (RNN)
 - ✓ Identifier les applications en électronique

►5.8.1 Deep Learning vs Machine Learning

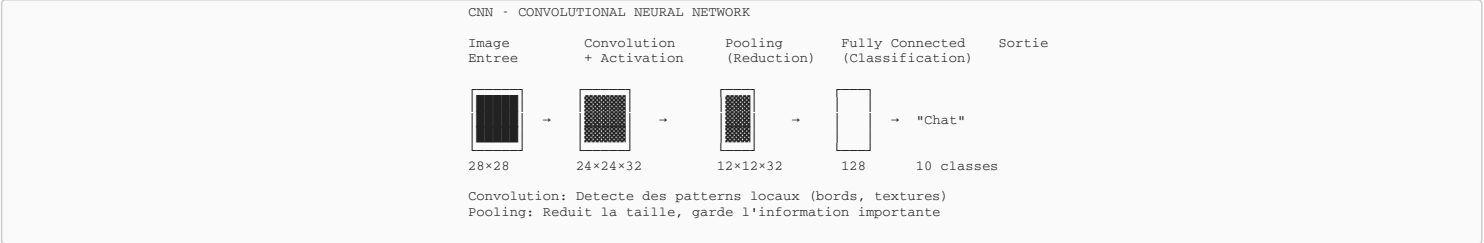
 **Deep Learning (Apprentissage Profond)**

Sous-domaine du ML utilisant des reseaux de neurones avec **plusieurs couches cachees** (profonds). Capable d'apprendre automatiquement des representations hierarchiques complexes a partir de donnees brutes.

Aspect	Machine Learning Classique	Deep Learning
Features	Extraction manuelle (humain)	Extraction automatique (reseau)
Donnees	Fonctionne avec peu de donnees	Necessite beaucoup de donnees
Materiel	CPU suffisant	GPU recommande
Interpretabilite	Souvent interpretable	Boite noire

►5.8.2 Reseaux Convolutifs (CNN)

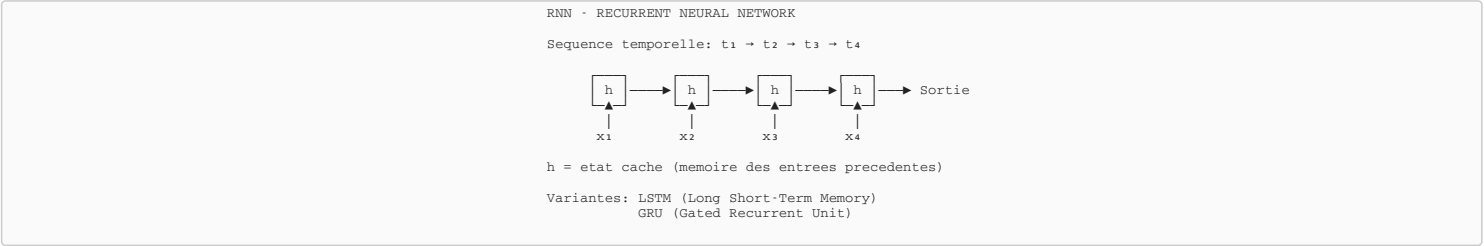
Specialises dans le traitement d'**images**. Apprennent des filtres (features) hierarchiques.



- Applications des CNN**
- Classification d'images (chat vs chien)
 - Detection d'objets (YOLO, SSD)
 - Reconnaissance faciale
 - Inspection visuelle de PCB
 - Lecture de compteurs analogiques

►5.8.3 Reseaux Recurrents (RNN)

Specialises dans les **sequences** (texte, audio, series temporelles). Possedent une "memoire".



- Applications des RNN/LSTM**
- Prediction de series temporelles (capteurs)
 - Reconnaissance vocale
 - Traduction automatique
 - Detection d'anomalies temporelles

🐍 **5.5.9 - Python pour le Machine Learning**
Bibliothèques essentielles et implémentation pratique

►Environnement Python ML

Python est le langage de référence pour le Machine Learning grâce à son écosystème riche de bibliothèques spécialisées.

Installation des bibliothèques essentielles
\$ pip install numpy pandas matplotlib scikit-learn

Bibliothèque	Rôle
NumPy	Calcul matriciel et opérations numériques
Pandas	Manipulation de données tabulaires (DataFrame)
Matplotlib	Visualisation et graphiques
Scikit-learn	Algorithmes ML (classification, régression, clustering)

►Exemple Complet : Classification avec Scikit-learn

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Charger les données
from sklearn.datasets import load_iris
data = load_iris()
X, y = data.data, data.target

# 2. Séparer train/test (80%/20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 3. Entraîner le modèle KNN
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# 4. Prédire et évaluer
y_pred = model.predict(X_test)
print(f"Précision : {accuracy_score(y_test, y_pred)*100:.1f}%")
```

5.5.10 - TensorFlow et Réseaux de Neurones

Framework de Deep Learning de Google

►Introduction à TensorFlow/Keras

TensorFlow est le framework de deep learning le plus utilisé, développé par Google. **Keras** (intégré dans TensorFlow) offre une API simplifiée pour construire des réseaux de neurones.

```
$ pip install tensorflow
```

►Exemple : Réseau de Neurones pour Classification

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Construire le modèle
model = models.Sequential([
    layers.Dense(64, activation='relu', input_shape=(4,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(3, activation='softmax') # 3 classes
])

# Compiler
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Entraîner
model.fit(X_train, y_train, epochs=50, validation_split=0.2)

# Évaluer
loss, acc = model.evaluate(X_test, y_test)
print(f"Précision : {acc*100:.1f}%")
```

5.11 - TinyML : IA Embarquée sur Microcontrôleurs

Déployer des modèles ML sur ESP32 et STM32

►Qu'est-ce que TinyML ?

TinyML = Machine Learning sur microcontrôleurs (MCU) avec très peu de ressources (< 256 KB RAM). Permet l'inférence IA directement sur le capteur, sans connexion cloud.

Avantages

- **Latence ultra-faible** : pas de réseau, réponse immédiate
- **Confidentialité** : données traitées localement
- **Faible consommation** : fonctionne sur batterie
- **Autonomie** : fonctionne sans Internet

Applications BTS Électronique

Détection d'anomalies vibratoires (maintenance prédictive), reconnaissance vocale (commandes "ON"/"OFF"), détection de mouvement (accéléromètre), classification de sons (alerte intrusion).

►Workflow TinyML

1. **Collecter** des données sur le capteur cible
2. **Entraîner** un modèle en Python (TensorFlow/Keras)
3. **Convertir** avec TensorFlow Lite : `converter = tf.lite.TFLiteConverter.from_saved_model(model)`
4. **Quantifier** en INT8 pour réduire la taille (+4)
5. **Déployer** sur ESP32/STM32 avec TensorFlow Lite Micro

5.12 - Projets IA pour BTS Électronique

Applications concrètes et idées de projets

►Projets Réalisables

Projet	Technologie	Difficulté
Détection anomalies capteur vibration	Scikit-learn + ESP32	☆☆
Reconnaissance chiffres manuscrits (MNIST)	TensorFlow CNN	☆☆
Commande vocale ON/OFF embarquée	TinyML + ESP32	☆☆☆
Contrôle qualité visuel PCB	CNN + caméra Pi	☆☆☆
Prédiction consommation énergétique	Régression + MQTT	☆☆

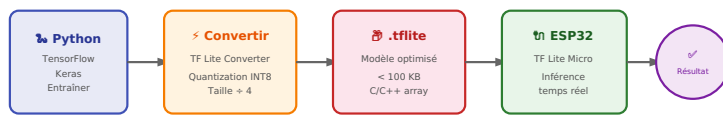


Figure - Workflow TinyML : du modèle Python au microcontrôleur

🌐 Chapitre 6 - Internet des Objets (IoT)

ESP32 · MQTT · Node-RED · Dashboards · Android · App Inventor

🌐 6.6.1 - Structure d'un Systeme IoT

Architecture et composants fondamentaux

🕒 Objectifs du chapitre

- ✓ Définir l'Internet des Objets et ses caracteristiques
- ✓ Comprendre l'architecture en couches d'un systeme IoT
- ✓ Identifier les domaines d'application
- ✓ Connaitre les flux de donnees typiques

►6.1.1 Definition de l'Internet des Objets

🗨️Definition IoT

L'

Internet des Objets

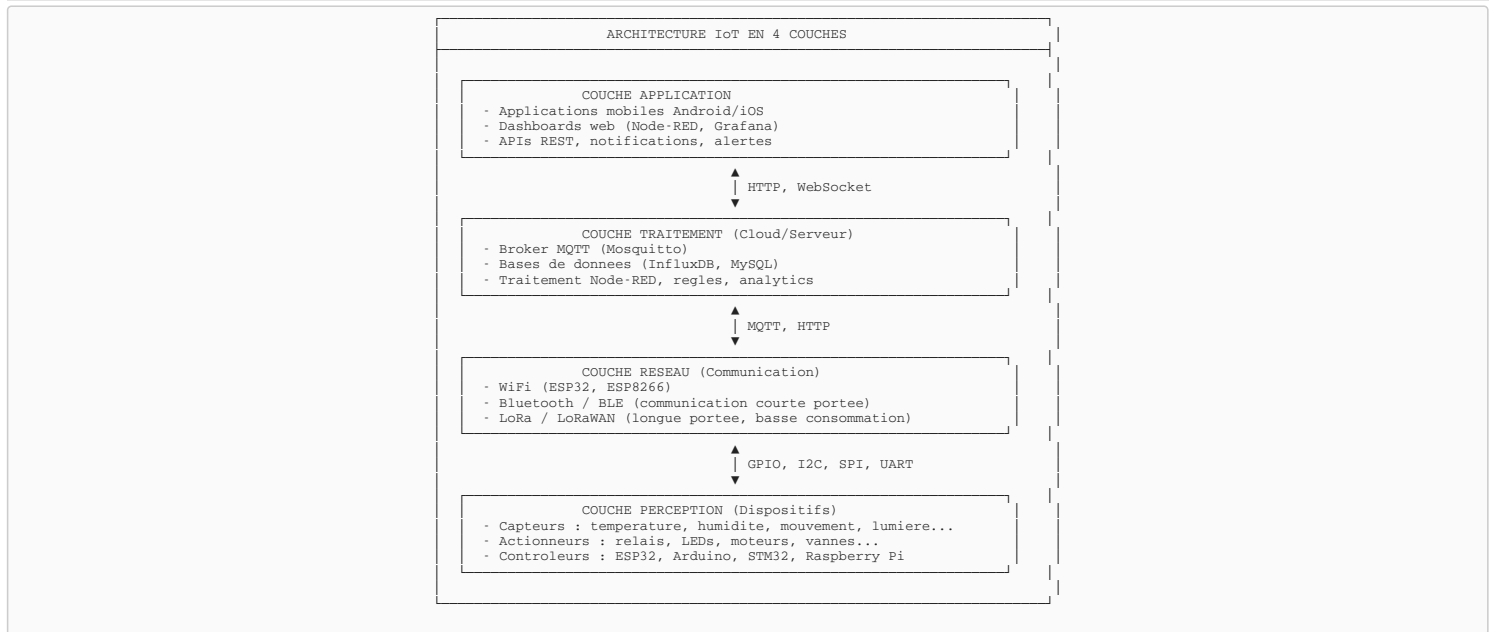
(IoT - Internet of Things) designe l'interconnexion entre Internet et des objets physiques capables de collecter, transmettre et recevoir des donnees via des reseaux de communication. Ces objets "intelligents" peuvent interagir avec leur environnement et communiquer entre eux ou avec des systemes cloud.

Caracteristiques d'un objet connecte

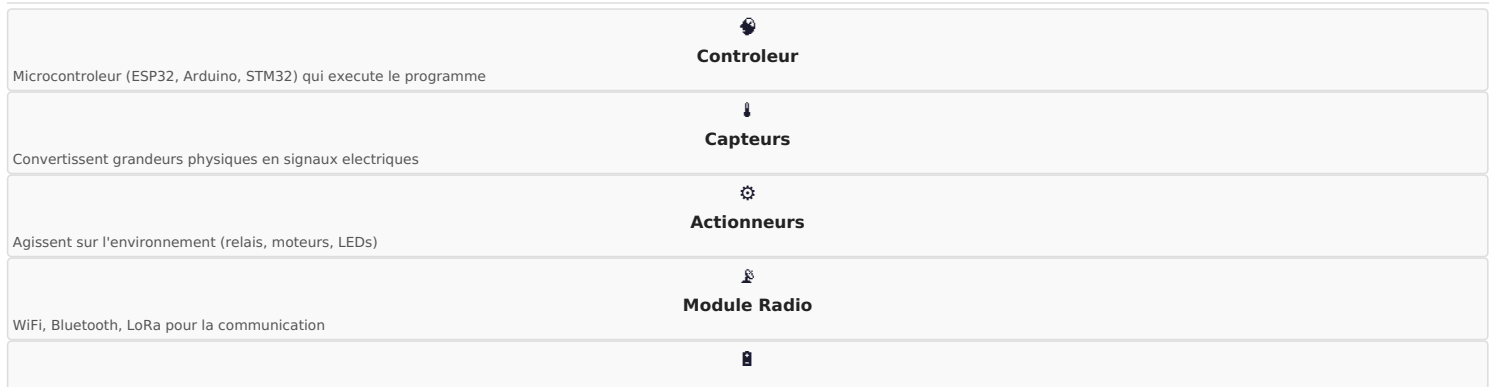
- **Identifiable** : Possede une identite unique (adresse IP, MAC, UUID)
- **Connecte** : Communique via un reseau (WiFi, Bluetooth, LoRa...)
- **Intelligent** : Embarque une capacite de traitement (microcontroleur)
- **Interactif** : Percoit (capteurs) et agit (actionneurs) sur l'environnement
- **Autonome** : Fonctionne de maniere independante, souvent sur batterie

	 75 Mds
Objets connectes prevus en 2025	
	 1100 Mds \$
Marche mondial IoT	
	 40%
IoT industriel (IIoT)	
	 25%
Smart Home	

►6.1.2 Architecture d'un Systeme IoT



►6.1.3 Composants d'un Dispositif IoT



	Alimentation
Batterie, USB, secteur selon l'application	
	
Memoire	
Flash, EEPROM pour stocker donnees et config	

Flux de donnees typique

```
/* Flux de donnees dans un systeme IoT */

1. ACQUISITION
  Capteur DHT22 → Mesure temperature 25.5°C → Signal numerique

2. TRAITEMENT LOCAL (ESP32)
  Lecture capteur → Formatage JSON → {"temp": 25.5, "hum": 60}

3. TRANSMISSION (WiFi + MQTT)
  ESP32 → Publish topic "maison/salon/capteur" → Broker Mosquitto

4. TRAITEMENT SERVEUR (Node-RED)
  Subscribe topic → Regles → Si temp > 30 → Alerte

5. STOCKAGE
  Node-RED → InfluxDB → Historique des mesures

6. VISUALISATION (Dashboard)
  Node-RED Dashboard → Graphiques temps reel → Navigateur/Mobile

7. ACTION (retour vers ESP32)
  Utilisateur clique "Ventilateur ON" → MQTT → ESP32 → Relais ON
```

►6.1.4 Domaines d'Application

Domaine	Applications	Exemples
Smart Home	Domotique, securite, confort	Thermostat Nest, éclairage Philips Hue
Industrie 4.0	Maintenance predictive, suivi production	Capteurs vibration, OEE temps reel
Agriculture	Irrigation, monitoring cultures	Capteurs sol, stations meteo
Sante	Wearables, telemedecine	Montres connectees, oxymetres
Smart City	Eclairage, parking, dechets	Lampadaires intelligents, capteurs niveau
Transport	Flottes, vehicules connectes	GPS trackers, OBD-II

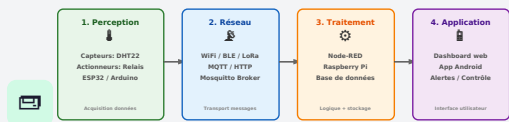


Figure - Architecture IoT en 4 couches

6.2 – Controleurs IoT

ESP32 et plateformes de developpement

🎯 Objectifs du chapitre

✓

 Connaitre les principaux controleurs IoT

✓

 Maitriser l'ESP32 et ses caracteristiques

✓

 Configurer l'environnement Arduino IDE

✓

 Programmer l'ESP32 pour l'IoT

►6.2.1 Comparaison des Controleurs IoT

Controleur	CPU	WiFi	Bluetooth	GPIO	Prix
ESP32	Dual-Core 240MHz	✓	✓ BLE	34	~5€
ESP8266	Single 80MHz	✓	X	17	~3€
Arduino Uno	ATmega328 16MHz	Shield	Shield	14	~20€
Raspberry Pi Pico W	Dual-Core 133MHz	✓	✓	26	~8€
STM32 + Module	ARM Cortex-M 168MHz	Module	Module	80+	~15€

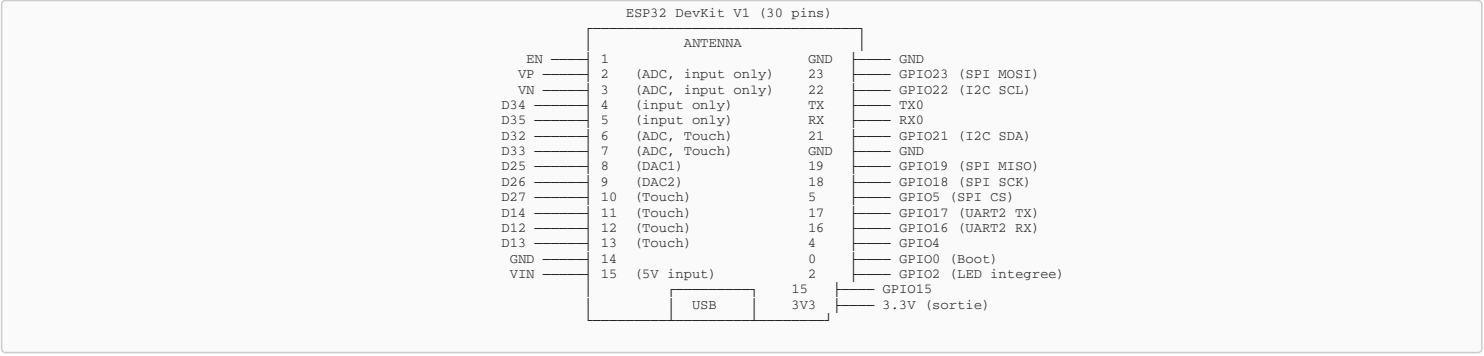
💡Choix recommande : ESP32

L'ESP32 est le controleur ideal pour l'IoT : WiFi + Bluetooth integres, puissant, economique, excellente documentation et communaute active.

►6.2.2 Caracteristiques de l'ESP32

	 Dual-Core
Xtensa LX6 240 MHz	
	 WiFi
802.11 b/g/n 2.4GHz	
	 Bluetooth
Classic + BLE 4.2	
	 520 Ko SRAM
+ 4-16 Mo Flash	
	 34 GPIO
ADC, DAC, PWM, I2C, SPI	
	 Deep Sleep
10 µA consommation	

Brochage ESP32 DevKit



►6.2.3 Configuration Arduino IDE pour ESP32

```
/* Installation ESP32 dans Arduino IDE */

1. Ouvrir Arduino IDE
2. Fichier → Preferences
3. Dans "URL de gestionnaire de cartes supplementaires", ajouter :
   https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

4. Outils → Type de carte → Gestionnaire de cartes
5. Rechercher "esp32"
6. Installer "ESP32 by Espressif Systems"

7. Selectionner : Outils → Type de carte → ESP32 Dev Module
8. Selectionner le port COM (ex: COM3, /dev/ttyUSB0)
9. Upload Speed : 115200
```

Premier programme : Blink LED

```
/* Clignotement LED integree ESP32 */

#define LED_PIN 2 // LED integree sur GPIO2

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("ESP32 démarre!");
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  Serial.println("LED ON");
  delay(1000);

  digitalWrite(LED_PIN, LOW);
  Serial.println("LED OFF");
  delay(1000);
}
```

►6.2.4 Lecture de Capteurs avec ESP32

```
/* Lecture capteur DHT22 (temperature + humidite) */

#include "DHT.h"

#define DHTPIN 4 // GPIO4
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("Capteur DHT22 initialise");
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Erreur lecture capteur!");
    return;
  }

  Serial.printf("Temperature: %.1f°C\n", temperature);
  Serial.printf("Humidite: %.1f%%\n", humidity);

  delay(2000); // DHT22 necessite 2s entre lectures
}
```

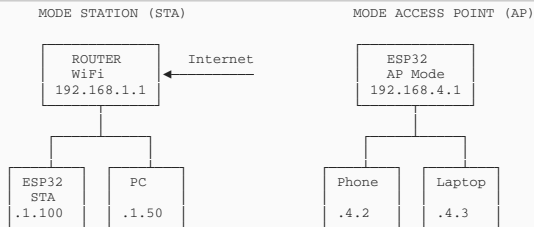
📶 6.6.3 - Communication WiFi
Connexion sans fil 2.4 GHz

🎯 Objectifs du chapitre

- ✓ Comprendre les modes WiFi (Station, AP)
- ✓ Connecter l'ESP32 a un reseau WiFi
- ✓ Creer un point d'accès WiFi
- ✓ Gerer la reconnexion automatique

►6.3.1 Modes WiFi ESP32

Mode	Description	Usage
WIFI_STA	Station : se connecte a un routeur	Acces Internet, cloud, MQTT
WIFI_AP	Access Point : cree son reseau	Configuration, reseau local isole
WIFI_AP_STA	Les deux simultanement	Repeteur, config + cloud



►6.3.2 Connexion Mode Station

```

/* Connexion WiFi en mode Station */

#include <WiFi.h>

const char* ssid = "NomDuReseau";
const char* password = "MotDePasse";

void setup() {
    Serial.begin(115200);

    // Demarrer la connexion WiFi
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    Serial.print("Connexion a ");
    Serial.print(ssid);

    // Attendre la connexion
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi connecte!");
    Serial.print("Adresse IP: ");
    Serial.println(WiFi.localIP());
    Serial.print("Force signal (RSSI): ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
}

void loop() {
    // Verifier la connexion
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi perdu! Reconnexion...");
        WiFi.reconnect();
    }
    delay(10000);
}

```

►6.3.3 Mode Access Point

```

/* Creer un point d'accès WiFi */

#include <WiFi.h>

const char* ap_ssid = "ESP32_IoT";
const char* ap_password = "12345678"; // Min 8 caracteres

void setup() {
    Serial.begin(115200);

    // Configurer l'Access Point
    WiFi.mode(WIFI_AP);
    WiFi.softAP(ap_ssid, ap_password);

    Serial.println("Point d'accès cree!");
    Serial.print("SSID: ");
    Serial.println(ap_ssid);
    Serial.print("IP: ");
    Serial.println(WiFi.softAPIP()); // 192.168.4.1
}

void loop() {
    Serial.printf("Clients connectes: %d\n",
                  WiFi.softAPgetStationNum());
    delay(5000);
}

```



6.6.4 - Communication Bluetooth

Bluetooth Classic et BLE pour l'IoT

🕒 Objectifs du chapitre

- ✓ Differencier Bluetooth Classic et BLE
- ✓ Creer une communication serie Bluetooth
- ✓ Comprendre l'architecture GATT du BLE
- ✓ Communiquer avec une application Android

►6.4.1 Bluetooth Classic vs BLE

Caracteristique	Bluetooth Classic	Bluetooth Low Energy
Debit	1-3 Mbps	125 kbps - 2 Mbps
Consommation	~30 mA	< 15 mA (pics courts)
Portee	~100 m	~100 m (BLE 5: 400 m)
Latence	~100 ms	~6 ms
Appairage	Obligatoire	Optionnel
Usage	Audio, fichiers	Capteurs IoT, beacons

►6.4.2 Bluetooth Serial (SPP)

Le mode **Bluetooth Serial** emule une liaison serie RS232 via Bluetooth, permettant une communication simple avec un smartphone.

```
/* Bluetooth Serial avec ESP32 */

#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

void setup() {
  Serial.begin(115200);
  pinMode(2, OUTPUT); // LED

  // Demarrer Bluetooth avec nom visible
  SerialBT.begin("ESP32_BT");
  Serial.println("Bluetooth démarre. Appairez votre telephone!");
}

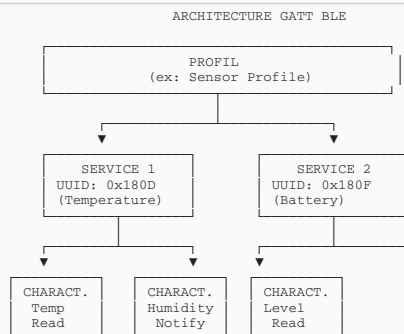
void loop() {
  // Reception depuis smartphone
  if (SerialBT.available()) {
    char c = SerialBT.read();
    Serial.write(c); // Afficher sur Serial

    // Commandes simples
    if (c == '1') {
      digitalWrite(2, HIGH);
      SerialBT.println("LED ON");
    } else if (c == '0') {
      digitalWrite(2, LOW);
      SerialBT.println("LED OFF");
    }
  }

  // Envoi depuis Serial vers smartphone
  if (Serial.available()) {
    SerialBT.write(Serial.read());
  }
}
```

►6.4.3 Bluetooth Low Energy (BLE)

Le BLE utilise une architecture **GATT** (Generic Attribute Profile) avec des Services et Caracteristiques :



```
/* Serveur BLE avec ESP32 */

#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID          "4fa1c201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID   "beb5483e-36e1-4688-b7f5-ea07361b26a8"

BLECharacteristic* pCharacteristic;
bool deviceConnected = false;

class MyCallbacks: public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) { deviceConnected = true; }
  void onDisconnect(BLEServer* pServer) { deviceConnected = false; }
};

void setup() {
  BLEDevice::init("ESP32_BLE");
  BLEServer* pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyCallbacks());

  BLEService* pService = pServer->createService(SERVICE_UUID);
  pCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
  );
  pCharacteristic->addDescriptor(new BLE2902());

  pService->start();
  pServer->getAdvertising()->start();
}

void loop() {
  if (deviceConnected) {
    float temp = 25.5; // Lecture capteur
    char buf[8];
    sprintf(buf, "%.1f", temp);
    pCharacteristic->setValue(buf);
    pCharacteristic->notify();
  }
  delay(2000);
}
```



6.6.5 - Communication LoRa

Longue portee, basse consommation (LPWAN)

- 🕒 **Objectifs du chapitre**
- ✓ Comprendre la technologie LoRa et LPWAN
 - ✓ Connaitre les parametres de modulation
 - ✓ Configurer un module LoRa avec ESP32
 - ✓ Comparer LoRa avec WiFi et Bluetooth

►6.5.1 Qu'est-ce que LoRa ?

📡 **LoRa - Long Range**
Technologie de modulation radio developpee par Semtech, utilisant la modulation **CSS** (Chirp Spread Spectrum). Ideale pour l'IoT distant avec faible debit et tres longue autonomie.

	📶 15+ km
Portee en zone rurale	
	🔋 10 ans
Autonomie sur pile	
	📶 50 kbps
Debit maximum	

►6.5.2 Comparaison des Technologies Sans Fil

Technologie	Portee	Debit	Consommation	Usage
WiFi	~100 m	Jusqu'a Gbps	Elevee	Internet, streaming
Bluetooth	~100 m	1-3 Mbps	Moyenne	Peripheriques, audio
BLE	~100 m	2 Mbps	Tres faible	Capteurs, wearables
LoRa	15+ km	50 kbps	Tres faible	Agriculture, smart city
Zigbee	~100 m	250 kbps	Faible	Domotique, mesh

►6.5.3 Communication LoRa Point-a-Point

```
/* Emetteur LoRa avec ESP32 + SX1276 */

#include <SPI.h>
#include <LoRa.h>

// Pins pour module LoRa
#define SS 18
#define RST 14
#define DIO0 26
#define BAND 868E6 // Frequence Europe (868 MHz)

void setup() {
  Serial.begin(115200);
  LoRa.setPins(SS, RST, DIO0);

  if (!LoRa.begin(BAND)) {
    Serial.println("Erreur initialisation LoRa!");
    while(1);
  }

  // Configuration
  LoRa.setSpreadingFactor(9); // SF7-SF12
  LoRa.setSignalBandwidth(125E3); // 125 kHz
  LoRa.setTxPower(14); // 14 dBm

  Serial.println("LoRa initialise!");
}

void loop() {
  // Envoi d'un paquet
  LoRa.beginPacket();
  LoRa.print("Hello LoRa #");
  LoRa.print(millis());
  LoRa.endPacket();

  Serial.println("Paquet envoye");
  delay(5000);
}
```

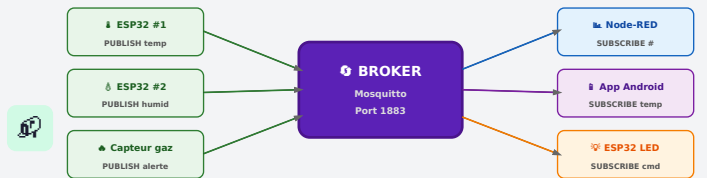


Figure - Architecture MQTT : Publish/Subscribe via Broker Mosquitto

6.6 - Protocole MQTT

Messagerie legere pour l'IoT

- 🕒 **Objectifs du chapitre**
- ✓ Comprendre l'architecture publish/subscribe
 - ✓ Maitriser les concepts MQTT (topics, QoS, retain)
 - ✓ Programmer un client MQTT sur ESP32
 - ✓ Publier et s'abonner a des topics

►6.6.1 Architecture MQTT

MQTT - Message Queuing Telemetry Transport

Protocole de messagerie

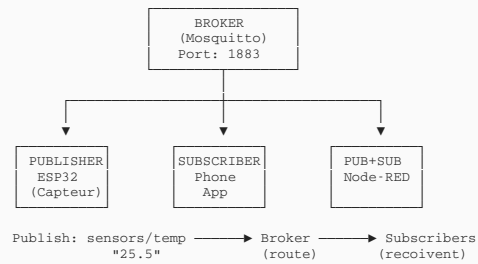
publish/subscribe

leger, concu pour les reseaux a faible bande passante et les appareils a ressources limitees. Port standard :

1883

(8883 avec TLS).

ARCHITECTURE MQTT



►6.6.2 Topics MQTT

Les **topics** sont des chemins hierarchiques utilises pour organiser les messages :

```
/* Structure des topics MQTT */

// Hierarchie avec "/"
maison/salon/temperature
maison/salon/humidite
maison/cuisine/lumiere
usine/machine1/vibration

/* Wildcards pour abonnements */

+ : Remplace UN niveau
maison+/temperature → salon, cuisine, chambre...

# : Remplace TOUS les niveaux (fin uniquement)
maison/# → tout ce qui commence par maison/
```

Qualite de Service (QoS)

QoS	Garantie	Usage
0	At most once (peut etre perdu)	Donnees frequentes (temperature)
1	At least once (doublons possibles)	Donnees importantes
2	Exactly once	Commandes critiques

►6.6.3 Client MQTT ESP32

```

/* Client MQTT complet avec ESP32 */

#include <WiFi.h>
#include <PubSubClient.h>

// Configuration
const char* ssid = "VotreWiFi";
const char* password = "MotDePasse";
const char* mqtt_server = "192.168.1.100"; // IP du broker
const int mqtt_port = 1883;

WiFiClient espClient;
PubSubClient client(espClient);

// Callback reception messages
void callback(char* topic, byte* payload, unsigned int length) {
    Serial.printf("Message reçu [%s]: ", topic);

    String message;
    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }
    Serial.println(message);

    // Traiter les commandes
    if (String(topic) == "esp32/led") {
        if (message == "ON") digitalWrite(2, HIGH);
        if (message == "OFF") digitalWrite(2, LOW);
    }
}

void reconnect() {
    while (!client.connected()) {
        Serial.print("Connexion MQTT...");
        if (client.connect("ESP32Client")) {
            Serial.println("connecte!");
            client.subscribe("esp32/led"); // S'abonner
        } else {
            Serial.println(" echec, retry...");
            delay(5000);
        }
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(2, OUTPUT);

    // Connexion WiFi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) delay(500);
    Serial.println("WiFi OK");

    // Configuration MQTT
    client.setServer(mqtt_server, mqtt_port);
    client.setCallback(callback);
}

void loop() {
    if (!client.connected()) reconnect();
    client.loop();

    // Publier toutes les 10 secondes
    static unsigned long lastMsg = 0;
    if (millis() - lastMsg > 10000) {
        lastMsg = millis();

        float temp = 25.5; // Lecture capteur
        char msg[10];
        sprintf(msg, "%.1f", temp);

        client.publish("esp32/temperature", msg);
        Serial.printf("Publie: %s\n", msg);
    }
}

```



6.6.7 - Installation Broker Mosquitto

Mise en place du serveur MQTT

🕒 Objectifs du chapitre

- ✓ Installer Mosquitto sur différentes plateformes
- ✓ Configurer le broker MQTT
- ✓ Tester avec mosquitto_pub et mosquitto_sub
- ✓ Sécuriser le broker (authentification)

►6.7.1 Installation de Mosquitto

```

# Installation sur Raspberry Pi / Ubuntu / Debian
sudo apt update
sudo apt install mosquitto mosquitto-clients -y

# Demarrer et activer au démarrage
sudo systemctl start mosquitto
sudo systemctl enable mosquitto

# Verifier le statut
sudo systemctl status mosquitto

# Installation sur Windows
# 1. Télécharger depuis: https://mosquitto.org/download/
# 2. Executer l'installateur .exe
# 3. Le service démarre automatiquement

```

►6.7.2 Configuration Mosquitto

```
# Fichier: /etc/mosquitto/mosquitto.conf

# Ecouter sur toutes les interfaces
listener 1883

# Autoriser les connexions anonymes (dev uniquement)
allow_anonymous true

# --- Configuration avec authentification ---
# allow_anonymous false
# password_file /etc/mosquitto/passwd

# Creer un utilisateur :
sudo mosquitto_passwd -c /etc/mosquitto/passwd monuser
# Entrer le mot de passe quand demande

# Redemarrer apres modification
sudo systemctl restart mosquitto
```

►6.7.3 Test en Ligne de Commande

```
# Terminal 1 : S'abonner a un topic
mosquitto_sub -h localhost -t "test/topic"

# Terminal 2 : Publier un message
mosquitto_pub -h localhost -t "test/topic" -m "Hello MQTT!"

# S'abonner a TOUS les topics (debug)
mosquitto_sub -h localhost -t "#" -v

# Avec authentification
mosquitto_pub -h localhost -u monuser -P motdepasse -t "test" -m "Secure!"

# Connexion a un broker distant
mosquitto_sub -h broker.hivemq.com -t "test/#"
```

💡Brokers MQTT publics pour tests

- [broker.hivemq.com](#) - Port 1883
- [test.mosquitto.org](#) - Port 1883
- [broker.emqx.io](#) - Port 1883

⚠ Ne pas utiliser pour donnees sensibles !

📶 6.6.8 - Node-RED : Programmation Visuelle IoT

Flux de données, nœuds et déploiement

►Introduction à Node-RED

Node-RED est un outil de programmation visuelle open-source basé sur **Node.js**, développé par IBM. Il permet de créer des flux de données IoT en connectant des **nœuds** (nodes) entre eux par glisser-déposer, sans écrire de code complexe.

Caractéristiques principales

- **Interface web** accessible sur le port 1880
- **Nœuds visuels** : input, processing, output, dashboard
- **Protocoles supportés** : MQTT, HTTP, WebSocket, TCP, UDP
- **Extensible** : plus de 4000 nœuds additionnels via npm

►Installation sur Raspberry Pi

```
# Installation recommandée (script officiel)
$ bash <(curl -sL https://raw.githubusercontent.com/node-red/linux-installers/master/deb/update-nodejs-and-nodered)

# Démarrer Node-RED
$ node-red-start

# Activer au démarrage
$ sudo systemctl enable nodered

# Accéder à l'interface : http://IP_du_Pi:1880
```

►Concepts Fondamentaux

Concept	Description	Exemple
Nœud (Node)	Bloc fonctionnel unitaire	mqtt in, function, debug
Flux (Flow)	Ensemble de nœuds connectés	Capteur → Traitement → Dashboard
Message (msg)	Objet JSON circulant entre nœuds	<code>msg.payload = 25.5</code>
Palette	Bibliothèque de nœuds disponibles	node-red-dashboard, influxdb

Types de Nœuds

- **Input** (gauche) : mqtt in, http in, inject, serial in
- **Processing** (milieu) : function, switch, change, template
- **Output** (droite) : mqtt out, http response, debug, dashboard

►Exemple : Flux MQTT → Dashboard

```
// Nœud Function : traitement des données capteur
var data = JSON.parse(msg.payload);
msg.payload = {
  temperature: data.temp,
  humidite: data.hum,
  timestamp: new Date().toISOString()
};
return msg;
```

Flux type : [mqtt in: maison/salon/temp] → [json] → [function: traitement] → [dashboard: gauge + chart]

►Nœuds MQTT dans Node-RED

Configuration d'un nœud **mqtt in** :

- **Server** : localhost:1883 (Mosquitto local) ou broker.hivemq.com
- **Topic** : maison/salon/temperature
- **QoS** : 0 (at most once) / 1 (at least once) / 2 (exactly once)

- **Output** : auto-detect (string/JSON)



6.6.9 - Dashboards et Visualisation IoT

Création d'interfaces de monitoring en temps réel

►Node-RED Dashboard

Le module **node-red-dashboard** permet de créer des interfaces web de visualisation en temps réel directement depuis Node-RED.

```
# Installation du module dashboard
$ cd ~/.node-red
$ npm install node-red-dashboard

# Accéder au dashboard : http://IP_du_Pi:1880/ui
```

►Widgets Dashboard Disponibles

Widget	Usage	Exemple
Gauge	Jauge circulaire	Température actuelle
Chart	Graphique temporel	Historique température
Text	Affichage texte	Statut capteur
Button	Bouton interactif	Allumer/éteindre LED
Switch	Interrupteur ON/OFF	Contrôle relais
Slider	Curseur de valeur	Réglage PWM moteur
Notification	Alerte popup	Température critique !

►Organisation du Dashboard

Le dashboard est organisé en **Tabs** (onglets) > **Groups** (groupes) > **Widgets** :

- **Tab "Salon"** : Group "Température" (gauge + chart), Group "Éclairage" (switch + slider)
- **Tab "Jardin"** : Group "Arrosage" (button + text), Group "Météo" (chart)



Conseil :

Utiliser les thèmes sombre/clair du dashboard via le menu Settings. Ajouter un **notification node** pour les alertes critiques (température > seuil).

►Alternatives de Visualisation

- **Grafana** – Dashboards professionnels avec InfluxDB/Prometheus
- **ThingsBoard** – Plateforme IoT complète avec dashboards intégrés
- **Blynk** – Application mobile pour dashboards IoT



6.6.10 - Introduction à Android pour l'IoT

Développement d'applications mobiles IoT

►Android et l'IoT

Android est le système d'exploitation mobile le plus répandu au monde (+70% de parts de marché). Il offre un excellent support pour les communications IoT : WiFi, Bluetooth, BLE, HTTP, MQTT, WebSocket.

Outils de Développement

Outil	Niveau	Langage	Usage
MIT App Inventor	Débutant	Blocs visuels	Prototypage rapide
Android Studio	Intermédiaire	Java / Kotlin	Apps professionnelles
Flutter	Intermédiaire	Dart	Apps cross-platform
React Native	Intermédiaire	JavaScript	Apps hybrides

►Communication Android ↔ ESP32

Deux méthodes principales pour connecter une application Android à un système IoT :

Méthode 1 : Via Broker MQTT

Android → [MQTT publish] → **Broker Mosquitto** → [MQTT subscribe] → **ESP32**

Bibliothèque Android : [Eclipse Paho MQTT](#)

Méthode 2 : Bluetooth BLE Direct

Android → [BLE GATT] → **ESP32 BLE Server**

Idéal pour : contrôle local sans WiFi, faible consommation

Méthode 3 : HTTP REST API

Android → [HTTP GET/POST] → **ESP32 Web Server**

Simple mais nécessite que les deux soient sur le même réseau WiFi



6.11 - MIT App Inventor : Créer une App Android sans Code

Développement visuel d'applications IoT

►Qu'est-ce que MIT App Inventor ?

MIT App Inventor est un environnement de développement en ligne gratuit créé par le MIT. Il permet de créer des applications Android en assemblant des **blocs visuels** (comme Scratch), sans écrire de code.

Accès : ai2.appinventor.mit.edu (compte Google requis)

Deux modes de travail

- **Designer** : interface graphique – ajouter boutons, labels, sliders, images
- **Blocks** : logique de programmation – événements, conditions, boucles, communications

►Composants pour l'IoT

Composant	Usage IoT
Web (HTTP)	Requêtes GET/POST vers ESP32 Web Server

BluetoothClient	Communication Bluetooth Classic avec ESP32/Arduino
ActivityStarter	Lancer d'autres apps (MQTT Dash, etc.)
Clock (Timer)	Polling périodique des capteurs
Notifier	Alertes utilisateur (température critique)

►Exemple : App de Contrôle LED via Bluetooth

Designer : 2 boutons ("ON" / "OFF") + 1 label statut + 1 ListPicker (sélection Bluetooth)

Blocks :

- `ListPicker.BeforePicking` → Lister les appareils Bluetooth
- `ListPicker.AfterPicking` → `BluetoothClient.Connect(adresse)`
- `BoutonON.Click` → `BluetoothClient.SendText("1")`
- `BoutonOFF.Click` → `BluetoothClient.SendText("0")`

🔊Test :

Utiliser le

AI Companion

(app mobile MIT) pour tester en temps réel sur votre téléphone sans installer l'APK.

 6.12 - Projet Intégré : Application IoT Complète
ESP32 + MQTT + Node-RED + Android

►Architecture du Projet

Projet de **domotique** intégrant toutes les technologies étudiées :

- **ESP32** : lecture capteurs DHT22 (température/humidité) + contrôle relais
- **MQTT (Mosquitto)** : broker sur Raspberry Pi, topics structurés
- **Node-RED** : flux de traitement + dashboard web temps réel
- **App Android** : interface mobile de contrôle et monitoring

►Structure MQTT du Projet

```
// Topics MQTT structurés
maison/salon/temperature    → ESP32 publie (float)
maison/salon/humidite       → ESP32 publie (float)
maison/salon/lumiere/cmd     → App envoie "ON"/"OFF"
maison/salon/lumiere/status  → ESP32 publie état actuel
maison/cuisine/gaz/alerte    → ESP32 publie si détection
```

►Code ESP32 - Client MQTT Complet

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "DHT.h"

const char* ssid = "MonWiFi";
const char* mqtt_server = "192.168.1.50";
WiFiClient espClient;
PubSubClient client(espClient);
DHT dht(4, DHT22);

void callback(char* topic, byte* payload, unsigned int len) {
  String msg = String((char*)payload).substring(0, len);
  if (String(topic) == "maison/salon/lumiere/cmd") {
    digitalWrite(2, msg == "ON" ? HIGH : LOW);
    client.publish("maison/salon/lumiere/status", msg.c_str());
  }
}

void loop() {
  client.loop();
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  client.publish("maison/salon/temperature", String(t).c_str());
  client.publish("maison/salon/humidite", String(h).c_str());
  delay(5000);
}
```

►Flux Node-RED du Projet

Flux complet pour le dashboard de monitoring :

1. **[mqtt in]** topic: maison/# → reçoit toutes les données
2. **[switch]** → route selon le topic (température, humidité, alerte)
3. **[gauge]** → affiche température en temps réel
4. **[chart]** → historique sur 24h
5. **[notification]** → alerte si température > 35°C ou détection gaz
6. **[button]** → envoie ON/OFF sur topic lumière

►Résumé des Technologies

Couche	Technologie	Rôle
Perception	ESP32 + DHT22 + Relais	Acquisition données + actionneurs
Réseau	WiFi + MQTT	Transport des messages
Traitement	Mosquitto + Node-RED	Broker + logique + dashboard
Application	Dashboard web + App Android	Interface utilisateur

⚠Sécurité :

En production, toujours activer l'authentification MQTT (user/password), utiliser TLS/SSL pour le chiffrement, et ne pas exposer le broker sur Internet sans VPN. Ne pas utiliser pour des données sensibles sans précautions !

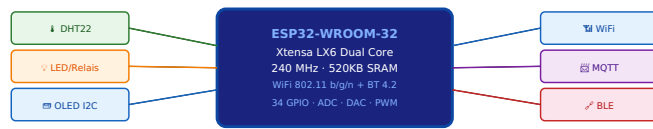


Figure - ESP32 : connexions capteurs et protocoles de communication

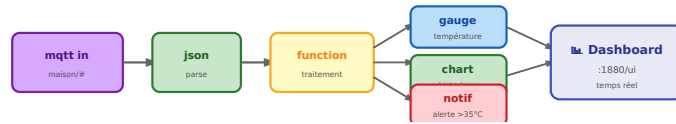


Figure - Flux Node-RED : MQTT → Traitement → Dashboard temps réel

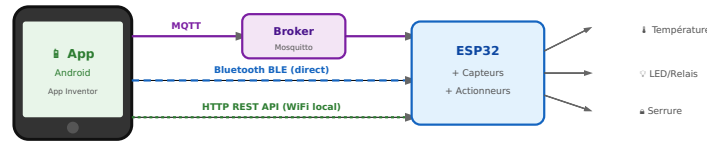


Figure - 3 méthodes de communication Android ↔ ESP32